

Double-click (or enter) to edit

```
# -*- coding: utf-8 -*-
"""
```

Лабораторная работа: Обработка признаков (Feature Engineering)

Датасет: Titanic (train.csv)

Задачи: Пропуски, Категории, Нормализация

```
"""
```

```
'\nЛабораторная работа: Обработка признаков (Feature Engineering)\nДатасет: Titanic (train.csv)\nЗадачи: Пропуски, Категории, Нормализация\n'
```

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler, OneHotEncoder,
from sklearn.impute import SimpleImputer
```

```
# --- 1. ЗАГРУЗКА ДАННЫХ ---
```

```
url = 'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic
```

```
df = pd.read_csv(url)
```

```
print("Исходный датасет:")
```

```
print(df.head())
```

```
print("\nИнформация о датасете:")
```

```
print(df.info())
```

Исходный датасет:

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

Информация о датасете:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PassengerId      891 non-null    int64
1   Survived         891 non-null    int64
2   Pclass           891 non-null    int64
3   Name             891 non-null    object
4   Sex              891 non-null    object
5   Age              714 non-null    float64
6   SibSp            891 non-null    int64
7   Parch            891 non-null    int64
8   Ticket           891 non-null    object
9   Fare             891 non-null    float64
10  Cabin            204 non-null    object
11  Embarked         889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
```

```
# --- 2. УСТРАНЕНИЕ ПРОПУСКОВ В ДАННЫХ (Task 1) ---
print("\n" + "="*50)
print("ЗАДАЧА 1: УСТРАНЕНИЕ ПРОПУСКОВ")
print("="*50)

print("Пропуски до обработки:")
print(df.isnull().sum())

df_clean = df.copy()

median_age = df_clean['Age'].median()
df_clean['Age'].fillna(median_age, inplace=True)
print(f"Пропуски в Age заполнены медианой: {median_age}")

mode_embarked = df_clean['Embarked'].mode()[0]
df_clean['Embarked'].fillna(mode_embarked, inplace=True)
print(f"Пропуски в Embarked заполнены модой: {mode_embarked}")

df_clean['Cabin'].fillna('Unknown', inplace=True)
df_clean.drop('Ticket', axis=1, inplace=True)

print("\nПропуски после обработки:")
print(df_clean.isnull().sum())
```

```
=====
ЗАДАЧА 1: УСТРАНЕНИЕ ПРОПУСКОВ
=====
Пропуски до обработки:
PassengerId      0
Survived          0
Pclass           0
```

```
Name      0
Sex       0
Age      177
SibSp     0
Parch     0
Ticket    0
Fare      0
Cabin     687
Embarked  2
```

```
dtype: int64
```

Пропуски в Age заполнены медианой: 28.0

Пропуски в Embarked заполнены модой: S

Пропуски после обработки:

```
PassengerId  0
Survived     0
Pclass       0
Name         0
Sex          0
Age          0
SibSp        0
Parch        0
Fare         0
Cabin        0
Embarked     0
```

```
dtype: int64
```

/tmp/ipykernel_1179/359015375.py:12: FutureWarning: A value is trying to be set on a copy of a DataFrame instance; using inplace=True will modify the DataFrame in place.
The behavior will change in pandas 3.0. This inplace method will never work because it is not implemented for this type of object.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.met

```
df_clean['Age'].fillna(median_age, inplace=True)
```

/tmp/ipykernel_1179/359015375.py:16: FutureWarning: A value is trying to be set on a copy of a DataFrame instance; using inplace=True will modify the DataFrame in place.
The behavior will change in pandas 3.0. This inplace method will never work because it is not implemented for this type of object.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.met

```
df_clean['Embarked'].fillna(mode_embarked, inplace=True)
```

/tmp/ipykernel_1179/359015375.py:19: FutureWarning: A value is trying to be set on a copy of a DataFrame instance; using inplace=True will modify the DataFrame in place.
The behavior will change in pandas 3.0. This inplace method will never work because it is not implemented for this type of object.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.met

```
df_clean['Cabin'].fillna('Unknown', inplace=True)
```

```
# --- 3. КОДИРОВАНИЕ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ (Task 2) ---
print("\n" + "="*50)
print("ЗАДАЧА 2: КОДИРОВАНИЕ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ")
print("="*50)
```

```
df_encoded = df_clean.copy()
```

```

le_sex = LabelEncoder()
df_encoded['Sex_Code'] = le_sex.fit_transform(df_encoded['Sex'])
print(f"Кодирование Sex: {dict(zip(le_sex.classes_, le_sex.transform(le_sex.classes_)))}")

embarked_dummies = pd.get_dummies(df_encoded['Embarked'], prefix='Embarked', drop_first=True)
df_encoded = pd.concat([df_encoded, embarked_dummies], axis=1)

df_encoded['Deck'] = df_encoded['Cabin'].astype(str).str[0]
deck_dummies = pd.get_dummies(df_encoded['Deck'], prefix='Deck', drop_first=True)
df_encoded = pd.concat([df_encoded, deck_dummies], axis=1)

df_encoded.drop(['Sex', 'Embarked', 'Cabin', 'Name', 'Deck'], axis=1, inplace=True)

print("\nДатасет после кодирования категорий (первые 5 колонок):")
print(df_encoded.head())
print(f"Размер датасета после кодирования: {df_encoded.shape}")

```

```
=====
```

ЗАДАЧА 2: КОДИРОВАНИЕ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ

```
=====
```

Кодирование Sex: {'female': np.int64(0), 'male': np.int64(1)}

Датасет после кодирования категорий (первые 5 колонок):

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare	Sex_Code	\
0	1	0	3	22.0	1	0	7.2500	1	
1	2	1	1	38.0	1	0	71.2833	0	
2	3	1	3	26.0	0	0	7.9250	0	
3	4	1	1	35.0	1	0	53.1000	0	
4	5	0	3	35.0	0	0	8.0500	1	

	Embarked_Q	Embarked_S	Deck_A	Deck_B	Deck_C	Deck_D	Deck_E	Deck_F	\
0	False	True	False	False	False	False	False	False	
1	False	False	False	False	True	False	False	False	
2	False	True	False	False	False	False	False	False	
3	False	True	False	False	True	False	False	False	
4	False	True	False	False	False	False	False	False	

	Deck_G	Deck_T	Deck_U
0	False	False	True
1	False	False	False
2	False	False	True
3	False	False	False
4	False	False	True

Размер датасета после кодирования: (891, 19)

```
# --- 4. НОРМАЛИЗАЦИЯ ЧИСЛОВЫХ ПРИЗНАКОВ (Task 3) ---
```

```
print("\n" + "="*50)
```

```
print("ЗАДАЧА 3: НОРМАЛИЗАЦИЯ / СТАНДАРТИЗАЦИЯ")
```

```
print("="*50)
```

```
numeric_features = ['Age', 'Fare']
```

```

df_scaled = df_encoded.copy()

print("Метод 1: Стандартизация (StandardScaler - Z-score)")
scaler_std = StandardScaler()
df_scaled[['Age_Standard', 'Fare_Standard']] = scaler_std.fit_transform(df_scaled[['Age', 'Fare']])

print("Метод 2: Min-Max Scaling (диапазон [0, 1])")
scaler_mm = MinMaxScaler()
df_scaled[['Age_MinMax', 'Fare_MinMax']] = scaler_mm.fit_transform(df_scaled[['Age', 'Fare']])

df_scaled.drop(numeric_features, axis=1, inplace=True)

print("\nРезультат нормализации (первые 10 строк):")
print(df_scaled[['Age_Standard', 'Fare_Standard', 'Age_MinMax', 'Fare_MinMax']])

print("\nСтатистика StandardScaler (Age_Standard):")
print(f"Среднее: {df_scaled['Age_Standard'].mean():.2f} (должен быть 0)")
print(f"Станд. отклонение: {df_scaled['Age_Standard'].std():.2f} (должно быть 1)")

print("\nСтатистика MinMaxScaler (Age_MinMax):")
print(f"Минимум: {df_scaled['Age_MinMax'].min():.2f} (должен быть 0)")
print(f"Максимум: {df_scaled['Age_MinMax'].max():.2f} (должен быть 1)")

print("\n" + "="*50)
print("ОБРАБОТКА ЗАВЕРШЕНА. ДАННЫЕ ГОТОВЫ ДЛЯ МОДЕЛИРОВАНИЯ.")
print("="*50)

```

=====

ЗАДАЧА 3: НОРМАЛИЗАЦИЯ / СТАНДАРТИЗАЦИЯ

=====

Метод 1: Стандартизация (StandardScaler - Z-score)
 Метод 2: Min-Max Scaling (диапазон [0, 1])

Результат нормализации (первые 10 строк):

	Age_Standard	Fare_Standard	Age_MinMax	Fare_MinMax
0	-0.565736	-0.502445	0.271174	0.014151
1	0.663861	0.786845	0.472229	0.139136
2	-0.258337	-0.488854	0.321438	0.015469
3	0.433312	0.420730	0.434531	0.103644
4	0.433312	-0.486337	0.434531	0.015713
5	-0.104637	-0.478116	0.346569	0.016510
6	1.893459	0.395814	0.673285	0.101229
7	-2.102733	-0.224083	0.019854	0.041136
8	-0.181487	-0.424256	0.334004	0.021731
9	-1.180535	-0.042956	0.170646	0.058694

Статистика StandardScaler (Age_Standard):

Среднее: 0.00 (должен быть 0)

Станд. отклонение: 1.00 (должно быть 1)

Статистика MinMaxScaler (Age_MinMax):

Минимум: 0.00 (должен быть 0)

Максимум: 1.00 (должен быть 1)

```
=====
ОБРАБОТКА ЗАВЕРШЕНА. ДАННЫЕ ГОТОВЫ ДЛЯ МОДЕЛИРОВАНИЯ.
=====
```